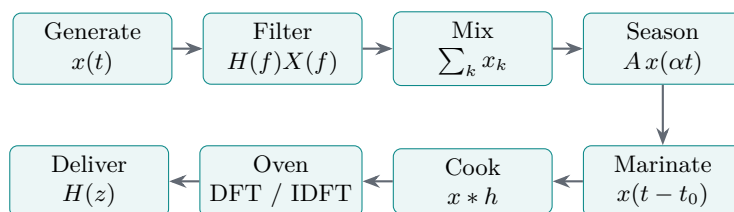

WaveKitchen / WaveBakery

Detailed Gameplay Overview

How the game plays, which files run each part, and how every station maps to Signals & Linear Systems



Contents

1	The game in one page	2
2	How the project is built	2
2.1	Two halves	2
2.2	Most important files	2
2.3	A signal inside the frontend	3
3	The player's journey	4
3.1	Screens around the kitchen	4
3.2	Difficulty and the timer	5
4	Station by station	5
4.1	Pantry and Generate Signal — ingredients are signals	5
4.2	Filtering Lab — washing is filtering	6
4.3	Mixing Lab — the bowl is superposition	7
4.4	Seasoning Lab — amplitude and time scaling	7
4.5	Marinating Lab — time shift	8
4.6	Caramelize Lab — amplitude modulation (Crème Brûlée, Feast)	8
4.7	Chop Lab — sampling, decimation and aliasing (Salad, Feast)	8
4.8	Cooking Lab — convolution with an appliance	9
4.9	Check Dish	9
4.10	Precision Oven — sampling theorem and the DFT	10
4.11	System Delivery — the z-plane	10
4.12	Score, Complete and the leaderboard	11
4.13	Signal Playground (Labs) — a sandbox	11
5	The recipes	12
5.1	Chef progression	12
6	Alignment with Signals & Linear Systems	12
7	Parts with little or no impact (flagged)	13
8	Glossary	14

1 The game in one page

WaveKitchen (the frontend calls itself *WaveBakery*) is a cooking game in which every culinary action is a signal-processing operation. Each **ingredient is a signal**, each **kitchen station is a system** that transforms the signal, and the **finished dish is the output** of the whole chain. The player's goal is to make their dish match a hidden *reference dish*: the output of the same chain run with the recipe's ideal parameters on clean ingredients.

Kitchen idea	Signal-processing meaning
Ingredient	A signal $x(t)$ (a periodic waveform, a parametric curve, or a recorded sound)
Dirty ingredient	Signal plus additive noise $x(t) + n(t)$
Washing	Frequency-selective filtering (FFT $\rightarrow$ mask $\rightarrow$ IFFT)
Mixing bowl	Superposition $\sum_k x_k(t)$ (linearity)
Seasoning	Amplitude scaling $Ax(t)$ and time scaling $x(\alpha t)$
Marinating	Time shift (delay) $x(t - t_0)$
Caramelizing	Amplitude modulation $x(t) [1 + m \cos(2\pi f_c t)]$
Chopping	Sampling / decimation and aliasing
Cooking appliance	An LTI system with impulse response $h[n]$; cooking is convolution $y = x * h$
Precision Oven	Sampling theorem, DFT/FFT, frequency-domain equalisation, IDFT reconstruction
Delivery cart	Discrete-time system $H(z)$: poles, zeros, stability, frequency response
Tasting table	Similarity metrics between the player's dish and the reference dish

A run is **timed** (5:00 on Easy down to 1:00 on MasterChef), stations are **unlocked in order**, and at the end the dish is **scored**, turned into **stars and points**, and placed on a **leaderboard**.

2 How the project is built

2.1 Two halves

Frontend (what the player sees and hears)

React 19 + TanStack Router/Start, built with Vite, written in TypeScript. It synthesises every ingredient signal mathematically, runs the whole per-station pipeline in the browser, draws every graph, plays every sound through the Web Audio API, and keeps the run state in `localStorage`. Folder: `frontend/src`.

Backend (the authoritative judge)

FastAPI + SQLAlchemy + SQLite, with NumPy/SciPy for DSP. It stores chefs, sessions, attempts and the leaderboard. On submission it *re-builds the dish from scratch* from the recorded parameters (it never trusts audio sent by the client) and scores it against the reference dish. Folder: `backend/app`.

Runner

`run.py` starts both servers with one command (backend on port 8000, Vite on port 5173, which proxies `/api` to the backend).

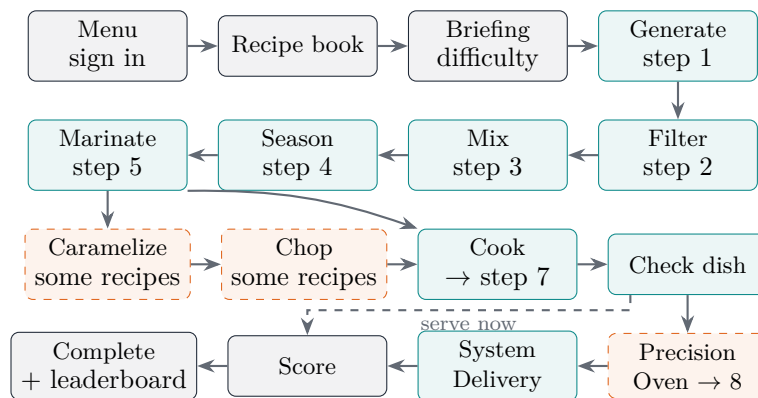
2.2 Most important files

File	Role
<code>frontend/src/lib/signals.ts</code>	Every ingredient's mathematical signal (equations, parametric curves, the recorded chicken), and the mixing-bowl superposition curve.
<code>frontend/src/lib/pipeline.ts</code>	The per-station DSP pipeline: mixing, seasoning, marinating, convolution, the reference dish, curve carrying, stage storage and invalidation.
<code>frontend/src/lib/dsp.ts</code>	FFT, low-pass filtering, convolution, and the similarity metric used for the final score.
<code>frontend/src/lib/precision-oven-dsp.ts</code>	Sampling/aliasing analysis, radix-2 FFT/IFFT, oven equaliser, reconstruction and oven metrics.
<code>frontend/src/lib/z-system.ts</code>	Delivery-cart systems: poles, zeros, gain, difference equation, stability score.
<code>frontend/src/lib/cooking-audio.ts</code>	The four cooking impulse responses $h[n]$ (grill, fry, bake, boil).
<code>frontend/src/lib/audio.ts</code>	Turns a 401-sample signal into audible sound.
<code>frontend/src/lib/recipes.ts</code>	Recipe book, difficulty/timer, run session, progression, backend sync and submission.
<code>frontend/src/lib/leaderboard.ts</code>	The on-device leaderboard.
<code>frontend/src/lib/api.ts</code>	HTTP client for the backend.
<code>frontend/src/routes/*.tsx</code>	One file per screen/station (Section 3).
<code>backend/app/dsp/core.py</code>	Backend DSP primitives: FFT, masks, filters, convolution, time operators, modulation, decimation.
<code>backend/app/dsp/instruments.py</code>	Backend ingredient synthesis (instrument-like voices).
<code>backend/app/dsp/contamination.py</code>	Seeded noise that makes ingredients “dirty”.
<code>backend/app/dsp/systems.py</code>	Appliance impulse responses (comb, decay, resonator, moving average, differencer).
<code>backend/app/dsp/pipeline.py</code>	The canonical backend pipeline <code>run_pipeline</code> .
<code>backend/app/dsp/metrics.py</code>	Prep (cleaning) score and dish score.
<code>backend/app/gameplay.py</code>	Judging, points, ranks, tier unlocks, System Delivery score.
<code>backend/app/routers/*.py</code>	HTTP endpoints: players, sessions, leaderboard, catalogue, direct DSP.
<code>backend/app/seed.py</code>	Ingredient, appliance and recipe catalogue (with targets).

2.3 A signal inside the frontend

Every stage passes around a `PipelineSignal` (defined in `frontend/src/lib/pipeline.ts`): **401 samples** spanning a *normalised one-second window*. Frequencies inside the game are therefore “game hertz”: cycles per window. A 4 Hz ingredient shows 4 cycles on screen. When a signal is played, `frontend/src/lib/audio.ts` repeats the window fast enough that its fundamental becomes an audible pitch between 110 and 880 Hz, so what you hear has the same shape as what you see.

3 The player's journey



Dashed orange boxes are optional or recipe-specific. Numbers are the progression levels stored in `localStorage (wavebakery_progress_<recipe>)`: a station stays locked (“Station Locked” screen) until the previous level is reached.

3.1 Screens around the kitchen

Screen	File	What happens
Landing / Menu	<code>routes/index.tsx</code> , <code>routes/menu.tsx</code>	Entry point; opens the chef sign-in (<code>components/game/ChefAuthModal.tsx</code>), leaderboard, settings, how-to-play and the Signal Playground.
Chef sign-in	<code>components/game/ChefAuthModal.tsx</code>	Register or sign in with name + password; the backend returns a token that identifies the chef on every later request.
Recipe book	<code>routes/recipe-book.tsx</code>	Choose one of 10 recipes; shows tier, difficulty and best score.
Briefing	<code>routes/briefing.tsx</code>	Shows the recipe’s ingredients, and lets the player choose difficulty (timer). Starting calls <code>startRecipeRun</code> , which resets progress, creates the run session and (if signed in) a backend game session.
Kitchen map / hub	<code>routes/kitchen.tsx</code> , <code>routes/kitchen-hub.tsx</code>	Station map and navigation hub.
Pantry	<code>routes/pantry.tsx</code>	Browse every ingredient and inspect its equation, waveform and spectrum (<code>components/game/IngredientSignalModal.tsx</code>).
Settings / How to play	<code>routes/settings.tsx</code> , <code>routes/how-to-play.tsx</code>	Theme, chef name, logout; the tutorial.
Leaderboard	<code>routes/leaderboard.tsx</code>	Server rankings per recipe/difficulty, global chef ranking, kitchen analytics, plus the on-device list when offline.

3.2 Difficulty and the timer

Difficulty	Time	Score multiplier	Source
Easy	5:00	$\times 0.8$	DIFFICULTY_CONFIGS in <code>lib/recipes.ts</code> , multipliers in <code>routes/score.tsx</code>
Medium	3:30	$\times 1.0$	
Hard	2:00	$\times 1.25$	
MasterChef	1:00	$\times 1.5$	

The timer (`useRecipeTimer`) counts down from the run's start. When it hits zero an expiry dialog (`components/game/RecipeTimer.tsx`) blocks the run and offers a retry; no completion score is awarded for an expired run. Time left at the end becomes a bonus (Section 4.12).

4 Station by station

Each station below lists what the player does, the files that implement it, the Signals & Systems concept it teaches, and why it exists and what it changes in the final result.

4.1 Pantry and Generate Signal — ingredients are signals

What the player does. Picks ingredients for the recipe, sees each ingredient's equation, waveform and spectrum, and listens to it. Finishing unlocks Filtering (or Mixing directly if nothing needs washing).

Files involved

`frontend/src/routes/generate.tsx` (station), `frontend/src/routes/pantry.tsx`, `frontend/src/lib/signals.ts` (all ingredient models), `getRecipeIngredientSamples` in `frontend/src/lib/pipeline.ts`, `frontend/src/lib/chicken-audio.ts` and `frontend/src/lib/chicken-samples.ts` (recorded chicken), `frontend/src/lib/audio.ts`.

The ingredients form a small catalogue of textbook signal families:

Ingredient	Family	Model (from <code>lib/signals.ts</code>)
Cheese	triangle	$x(t) = A \frac{2}{\pi} \arcsin(\sin(2\pi ft + \phi))$ — odd harmonics, $1/n^2$ roll-off
Sugar	square	$x(t) = A \operatorname{sgn}(\sin(2\pi ft + \phi))$ — odd harmonics, $1/n$ roll-off
Salt	square	same as sugar with small amplitude $A_s = 0.4$
Bread	superposition	$y(t) = 0.8 \sin(6\pi t) + 0.6 \sin(18\pi t)$ (3 Hz + 9 Hz)
Milk	sum of sines	$y = 1.4 \sin(0.7x) + 0.6 \sin(1.3x) + 0.3 \sin(2.1x + 0.8)$ (non-harmonic)
Flour	“flame”	$\operatorname{sq}(t) - \operatorname{sq}(2t) \cos(2\pi t)$ (product of signals)
Bun	dome	$5(0.5 + 0.5 \cos 0.3x)^{0.25} + 0.03 \cos 9x$
Butter	trapezoid	$\max(0, 1.2 - 1.2 \max(\operatorname{mod}(x + 9, 14) - 9 - 4, 0))$
Carrot	taper	$5 - (1 + \cos 0.5x)^4$
Cucumber	saturated	$6 \tanh(4 \cos x)$ — a soft-clipped cosine
Beef patty	parametric	$x = 2.5 \cos t + 5 \lfloor t/2\pi \rfloor, y = 0.7 \sin t$
Lettuce	parametric	$x = t, y = 2.2 \cos t + 0.45 \cos 7.5t$

Ingredient	Family	Model (from <code>lib/signals.ts</code>)
Noodles	parametric	$x = t + 3 \sin t$, $y = 5 \cos t$ (trochoid loops)
Tomato	parametric	$x = (2 + 0.2 \sin 3t) \cos t$, $y = -(1.5 + 0.2 \sin t) \sin t$
Onion	parametric	$x = (0.1 + 0.08t) \cos 3t$, $y = (0.1 + 0.08t) \sin 3t$ (spiral)
Sauce	parametric	$x = (3 + 2 \sin 5t) \cos t$, $y = (2 + \sin 5t) \sin t$
Egg	parametric	$x = (1.5 + 0.5 \sin t) \cos t$, $y = -(2 + 0.2 \sin t) \sin t$
Chicken	recorded	$s(t) = \text{PCM samples of public/sounds/chicken.wav (44.1 kHz)}$

A *parametric* ingredient is a 2-D curve $(x(t), y(t))$. For sound and for the 1-D pipeline the game uses its vertical component $y(t)$ as the time signal; for the mixing-bowl picture it keeps the full curve.

Signals & Systems concept

Continuous-time signal models and their classification: periodic vs aperiodic, sinusoids, square/triangle waves and their Fourier-series harmonic content, superposition of sinusoids, nonlinear shaping (saturation, clipping), parametric curves, and a real sampled recording (discrete-time signal at 44.1 kHz).

Why it is here & its impact

Every later station operates on these signals, so this is the input to the whole system. Choosing ingredients decides what goes into the bowl; a wrong or missing ingredient changes the mix and lowers the Mixing accuracy and the final similarity. Different families were chosen so the player can *see* why a square wave has a richer spectrum than a sine and why filtering affects them differently.

4.2 Filtering Lab — washing is filtering

What the player does. Each *washable* ingredient arrives with noise. The player drags a **cutoff frequency** slider over the spectrum, applies the filter and must reach a cleanliness of at least 85 % before moving on.

Files involved

```
frontend/src/routes/filtering.tsx; applyLowPassFilter, fft in
frontend/src/lib/dsp.ts; backend apply_filters / accept_ingredient in
backend/app/routers/sessions.py; dirty_signal, filtered_signal in
backend/app/gameplay.py; backend/app/dsp/contamination.py; apply_filter_chain in
backend/app/dsp/pipeline.py; prep_quality in backend/app/dsp/metrics.py.
```

When a backend session exists, the Filtering Lab shows the *server's* real dirty and clean signal for that ingredient, sends the chosen cutoff to the server (`filterIngredient`), and displays the server's filtered result and cleaning score. Offline, a local approximation is used. The filtered samples are saved and become the ingredient's input to Mixing.

Signals & Systems concept

Additive noise, the Fourier transform (FFT), the frequency domain, ideal vs practical low-pass filters (a raised-cosine transition band), filtering as multiplication in frequency $Y(f) = H(f)X(f)$, and the trade-off between removing noise and keeping the signal's own frequencies (cutting too low distorts the ingredient).

Why it is here & its impact

It is the first place the player must reason in the frequency domain. The server's cleaning ("prep") score is 28% of the authoritative dish score and is reported as the Filtering sub-score; the local filtering accuracy is one of the stage averages in the offline score. Over-filtering is detected and named in the feedback ("you cut real ingredient frequencies").

4.3 Mixing Lab — the bowl is superposition

What the player does. Drags ingredients from the tray into the bowl and presses *Mix*. The oscilloscope shows each ingredient and the combined signal.

Files involved

```
frontend/src/routes/mixing.tsx; computeMixedSignal, withMixCurve,
getMixingIngredientSamples in frontend/src/lib/pipeline.ts;
computeSuperpositionCurve, computeSuperpositionPath in
frontend/src/lib/signals.ts; backend bowl_signals, compute_stages in
backend/app/gameplay.py and stage_mix in backend/app/dsp/pipeline.py.
```

For ordinary ingredients the mix is the sample-by-sample sum. When a parametric ingredient is present the bowl draws the *2-D geometric* superposition $(\sum_k x_k(t), \sum_k y_k(t))$, which is why curves such as noodles make loops. That curve is carried as a visual track through every later station. The bowl contents are sent to the server, which mixes exactly those ingredients.

Signals & Systems concept

Linearity and the superposition principle; normalisation of a sum $(1/\sqrt{K})$ to keep amplitude comparable; the difference between adding signals in time and adding curves in the plane.

Why it is here & its impact

Mixing accuracy is the fraction of recipe ingredients correctly placed. On the server, missing or extra ingredients change the mixed signal and therefore the Mixing sub-score and every stage after it.

4.4 Seasoning Lab — amplitude and time scaling

What the player does. Sets *Signal strength* (amplitude A) and *Frequency character* (α) and watches the player curve approach the dashed target curve. Accuracy $\geq 85\%$ completes the station.

Files involved

```
frontend/src/routes/transform.tsx; computeSeasonedSignal, timeScaleSamples,
transformCurve in frontend/src/lib/pipeline.ts; backend stage_season and
stage_blend (time_scale) in backend/app/dsp/pipeline.py and
backend/app/dsp/core.py.
```

Signals & Systems concept

Amplitude scaling $y(t) = Ax(t)$ and time scaling $y(t) = x(\alpha t)$: $\alpha > 1$ compresses the signal (higher frequencies, more cycles), $\alpha < 1$ expands it. Time scaling scales the spectrum: $x(\alpha t) \leftrightarrow \frac{1}{|\alpha|}X(f/\alpha)$.

Why it is here & its impact

The dials are synced to the server (as `seasoning` and `blend`); a wrong gain or rate changes the reference comparison and triggers named feedback (“over-seasoned”, “over-blended”). The seasoning accuracy is a stage average and half of the “Transformation” score.

4.5 Marinating Lab — time shift

What the player does. Chooses a marinating time; the waveform is delayed (a flat lead-in, then the signal) and must match the target timing.

Files involved

`frontend/src/routes/marinate.tsx`; `computeMarinatedSignal`, `delayCurve` in `frontend/src/lib/pipeline.ts`; backend `stage_marinate` and `time_shift` (fractional delays via a linear phase ramp) in `backend/app/dsp/pipeline.py` and `backend/app/dsp/core.py`.

Signals & Systems concept

Time shifting $y(t) = x(t - t_0)$; causality and zero padding; the shift theorem $x(t - t_0) \leftrightarrow X(f)e^{-j2\pi f t_0}$, which the backend uses to delay by non-integer numbers of samples.

Why it is here & its impact

The time is synced to the server as `marinate` (seconds). It completes the Transformation score together with seasoning, and the server names mistakes (“marinated too long — excess time shift”).

4.6 Caramelize Lab — amplitude modulation (Crème Brûlée, Feast)

What the player does. Sets a carrier frequency f_c and modulation depth m and listens to the “caramelized” signal.

Files involved

`frontend/src/routes/caramelize.tsx`; backend `stage_caramelize` / `am_modulate` in `backend/app/dsp/pipeline.py` and `backend/app/dsp/core.py`; targets in `backend/app/seed.py`.

Signals & Systems concept

Amplitude modulation $y(t) = x(t)[1 + m \cos(2\pi f_c t)]$: the spectrum gains sidebands at $f \pm f_c$; the depth m sets their height.

Why it is here & its impact

Carrier and depth are sent to the server, whose pipeline modulates the dish before cooking; the recipe’s targets make the reference dish modulated too. The station also has its own accuracy, averaged into the local score for recipes that need it.

4.7 Chop Lab — sampling, decimation and aliasing (Salad, Feast)

What the player does. Chooses a decimation factor M and whether to use an anti-aliasing filter.

Files involved

frontend/src/routes/chop.tsx; backend stage_chop / decimate in backend/app/dsp/pipeline.py and backend/app/dsp/core.py.

Signals & Systems concept

Down-sampling by M (keep every M -th sample, zero-order hold back), the new Nyquist frequency $f_s/(2M)$, aliasing when content exceeds it, and the anti-aliasing low-pass filter that prevents it.

Why it is here & its impact

Factor and filter choice are sent to the server, which decimates for real before cooking; skipping the anti-alias filter is diagnosed (“Aliasing! You decimated without the anti-alias filter”).

4.8 Cooking Lab — convolution with an appliance

What the player does. Picks a cooking method (grill, fry, bake, boil — each is an impulse response) and slides the impulse across the signal to set the convolution depth. The panels show input x , impulse response h , output $x * h$ and the target.

Files involved

frontend/src/routes/cooking.tsx; computeConvolvedSignal in frontend/src/lib/pipeline.ts; kernels in frontend/src/lib/cooking-audio.ts; backend stage_cook, apply_system, cascade and the appliance impulse responses in backend/app/dsp/systems.py.

Signals & Systems concept

Linear time-invariant (LTI) systems are fully described by the impulse response; the output is the convolution $y[n] = \sum_k x[k] h[n - k]$. Different h give different characters: a comb of echoes (grill), a bright decaying burst (fry), a smooth long tail / low-pass (bake), a damped resonator (boil). A cascade of LTI systems is one LTI system with $h = h_1 * h_2 * \dots$.

Why it is here & its impact

The chosen appliance is the heart of the dish. The server’s Cooking sub-score now isolates this choice: it compares the player’s pre-cooking signal through the recipe’s appliances against the same signal through the player’s appliances, so the right appliance scores 100 even if earlier stages were imperfect.

4.9 Check Dish

What the player does. Inspects and listens to the cooked dish, then either serves it immediately or takes it to the Precision Oven (components/beamforming/ServeChoiceModal.tsx).

Files involved

frontend/src/routes/check-dish.tsx,
frontend/src/components/game/WaveformDisplay.tsx.

Why it is here & its impact

No computation of its own: it is a checkpoint that lets the player hear the result of convolution before the optional finishing stations.

4.10 Precision Oven — sampling theorem and the DFT

What the player does. Five steps:

1. **Sampling:** pick a sampling rate f_s and watch the reconstruction; below $2f_{\max}$ the dish aliases.
2. **Unlock:** type the minimum safe rate; it must satisfy $f_s \geq 2f_{\max}$ and be at most 64 Hz.
3. **FFT lab:** see the dish's spectrum next to the target's and tune three bands (Base Warmth 0–6 Hz, Crumb 6–16 Hz, Top Crisp 16–32 Hz), a browning cutoff and a charred-notch filter.
4. **IFFT:** rebuild the dish from the edited spectrum and listen.
5. **Plate:** save the result as the delivered dish.

Files involved

`frontend/src/routes/beam-delivery.tsx` (the Precision Oven screen; the file name is historical), `frontend/src/lib/precision-oven-dsp.ts`, `computeSignalSimilarity` in `frontend/src/lib/dsp.ts`.

Signals & Systems concept

The Nyquist–Shannon sampling theorem, aliasing and frequency folding, the DFT and its frequency axis $f_k = kf_s/N$, zero padding, the radix-2 Cooley–Tukey FFT, frequency-domain filtering, Hermitian symmetry of real signals, the inverse DFT, and band-limited (sinc/Dirichlet) reconstruction.

Why it is here & its impact

The oven score (spectrum match 40 %, waveform fidelity 50 %, sampling efficiency 10 %) becomes the Precision Oven Finishing accuracy, which counts for 10 % of the dish score on both the score screen and the server.

4.11 System Delivery — the z-plane

What the player does. Chooses the cart's "suspension" (a 1st-order low-pass, a 2nd-order resonator, a moving average or a notch), drags the pole on the z-plane (radius r , angle ω_0) and picks the sensor sampling rate. The screen shows poles/zeros, $|H(e^{j\omega})|$, phase, the dish spectrum before and after the cart, and an impulse-response animation; "Listen equalized dish" plays the dish filtered by the cart.

Files involved

`frontend/src/routes/system-delivery.tsx`,
`frontend/src/components/system/SystemResponsePlotter.tsx`,
`frontend/src/lib/z-system.ts`; backend `system_score_for` in `backend/app/gameplay.py`.

Signals & Systems concept

The z-transform, transfer functions $H(z) = g \prod (z - z_i) / \prod (z - p_j)$, difference equations, poles and zeros, BIBO stability (all poles inside $|z| = 1$), frequency response on the unit circle, FIR

(moving average) vs IIR (resonator, low-pass) systems.

Why it is here & its impact

A stable, alias-free cart scores 100 (unstable 15, and $\times 0.6$ below a 4 kHz sensor). The server recomputes this from the settings and counts it for 5 % of the dish score.

4.12 Score, Complete and the leaderboard

What the player sees. The score screen compares the player’s dish with the target (waveforms and a similarity percentage), shows the stage breakdown, stars, Chef Fourier’s feedback and a customer critique, and the total points. The Complete page repeats the score and stars; the leaderboard ranks the run.

Files involved

frontend/src/routes/score.tsx, frontend/src/routes/complete.tsx, frontend/src/lib/critiques.ts, frontend/src/lib/leaderboard.ts, frontend/src/routes/leaderboard.tsx; backend submit in backend/app/routers/sessions.py, judge, award, maybe_unlock in backend/app/gameplay.py, backend/app/dsp/metrics.py, backend/app/routers/leaderboard.py.

How it works in short:

1. The latest parameters are synced, then the run is submitted once.
2. The server rebuilds the player’s dish and the reference dish, computes a 0–100 score and stars (92/80/66/50/32 for 5–1 stars), points and possible tier unlocks. Offline, the browser computes a 0–100 score with the same weighting.
3. Total points on screen = score $\times 10 \times$ difficulty multiplier + a time bonus (2 per second left, at most 300, also $\times$ difficulty).
4. The run is saved to the on-device leaderboard; the server leaderboard keeps each chef’s best dish per recipe (filterable by difficulty).

Signals & Systems concept

Signal similarity measures: normalised cross-correlation, (normalised) mean-square error and SNR in dB, and spectral similarity (cosine similarity of dB magnitude spectra).

Why it is here & its impact

This is where every earlier decision becomes a number; the full formulas are in the *Detailed Architecture* document.

4.13 Signal Playground (Labs) — a sandbox

frontend/src/routes/labs.tsx offers free-play sandboxes: generator, filter, mixer, seasoning, marinating, convolution and a **beamforming lab** (phased speaker array, steering and windowing, frontend/src/lib/beamforming.ts and frontend/src/components/beamforming/*). Nothing here affects a recipe run or the score; it exists for exploration and teaching.

5 The recipes

Recipe	Tier	Ingredients	Season A, α	Marin.	Cook	Extra
Burger	1	Bun, Beef Patty, Cheese, Lettuce, Tomato, Salt	1.5, 0.8	1.25	grill	—
Sandwich	1	Bread, Chicken, Cheese, Lettuce, Tomato, Salt, Sauce	1.2, 1.1	0.85	grill	—
Golden Toast	1	Cheese	1.0, 1.0	1.0	bake	—
Cake	2	Flour, Egg, Butter, Sugar, Milk	1.8, 0.6	1.5	bake	—
Noodles	2	Noodles, Egg, Chicken, Onion, Salt	1.3, 1.2	1.75	boil	—
Velvet Soup	2	Onion, Salt, Butter	0.65, 0.7	1.05	boil	—
Crisp Salad	2	Lettuce, Tomato, Cucumber	0.9, 1.0	1.02	grill	Chop $M = 3$
Chicken Fry	3	Chicken, Flour, Egg, Salt, Butter	2.0, 1.4	1.4	fry	—
Crème Brûlée	3	Egg, Sugar, Milk, Butter	0.75, 0.85	1.16	bake	AM 180, 0.55
Grand Feast	4	Beef Patty, Cheese, Tomato, Lettuce, Bun	0.95, 1.25	1.2	grill	Chop $M = 2$; AM 240, 0.4

Targets live in `frontend/src/lib/recipes.ts` and, for the server, `backend/app/seed.py`; the two are kept identical.

5.1 Chef progression

- **Points:** a served dish earns $\lfloor \text{score} \times \text{tier} \times 1.5 \rfloor$ points, but only if it beats the chef's previous best on that recipe, and only on an unlocked tier (locked-tier runs are ranked but earn nothing).
- **Ranks** (by total points): Dishwasher 0, Prep Cook 250, Line Cook 600, Sous Chef 1100, Head Chef 1800, Signal Master 2600.
- **Tiers:** clearing 2 recipes of the current tier with a score ≥ 60 unlocks the next tier.

6 Alignment with Signals & Linear Systems

Concept	Where	Why it was added	Impact on the game
Signal classification, periodic signals, harmonics	Pantry, Generate	Ingredients must look and sound different for reasons the player can explain	Defines every input; wrong choice lowers Mixing
Fourier transform / spectrum	Filtering, Oven, Delivery	Noise is easier to see and remove in frequency	Drives the cleaning score (28 % server weight)
Filtering ($H(f)X(f)$)	Filtering	Washing = removing the noise band without cutting the ingredient	Prep score, over/under-filter feedback

Concept	Where	Why it was added	Impact on the game
Linearity / superposition	Mixing	A bowl is literally a sum	Mixing accuracy and server mix
Amplitude and time scaling	Seasoning	Two dials that change height and rate	Transformation score, server params
Time shift	Marinating	Delay is the simplest system	Transformation score, server params
Amplitude modulation	Caramelize	Sidebands as “browning”	Server pipeline (2 recipes)
Sampling, decimation, aliasing	Chop, Oven	Coarse chopping loses detail; Nyquist	Server pipeline (2 recipes); oven score
LTI systems, impulse response, convolution	Cooking	Each appliance is a system h	Central to the dish; Cooking score
DFT/FFT, IDFT, zero padding, reconstruction	Precision Oven	Finishing in the frequency domain	10 % of dish score
z-transform, poles/zeros, stability	System Delivery	A stable cart delivers the dish	5 % of dish score
Correlation, MSE/SNR, spectral similarity	Score	Objective comparison with the target	The final score, stars, leaderboard

7 Parts with little or no impact (flagged)

While mapping the project, the following parts were found to be unrelated to the Signals & Systems content or to have no effect on the game’s result.

▲ Flag: low or no impact

1. **Beamforming (Signal Playground only).** `frontend/src/lib/beamforming.ts`, `frontend/src/components/beamforming/BeamformingVisualizer.tsx`, `BeamPatternGraph.tsx`, `PhaseControls.tsx`, `SpeakerArray.tsx` are only used by the sandbox in `routes/labs.tsx`. `BeamTarget.tsx` is not used anywhere. Array processing is outside the core Signals & Linear Systems syllabus and has no effect on any score.
2. **Backend phased-array endpoint** `POST /api/sessions/{id}/beam-delivery` and `backend/app/dsp/beamforming.py`: never called by the frontend.
3. **Backend “direct DSP” endpoints** (`/api/dsp/generate`, `filter`, `mix`, `transform`, `convolve`, `beamforming`, `critique`) and their client methods in `lib/api.ts`: never called by the game.
4. **Unused client methods** in `lib/api.ts`: `getConvolution`, `getHealth`, `getPlayerHistory`, `getRecipes`, `getRecipe`, `getIngredients`, `getAppliances`, `loginPlayer`, `registerPlayer`.
5. **Unreachable noise types**: `corrupt` in `backend/app/dsp/contamination.py` only ever picks white, hiss or burst noise; its pink, hum and tone branches never run.

6. **WAV impulse-response loader** `loadCookingImpulseAudio` in `frontend/src/lib/cooking-audio.ts`: never called; cooking always uses the static kernels.
7. **Sensor sampling-rate toggle** on System Delivery: it changes the score factor and colours, but the plotted spectrum does not change with it.
8. **Check Dish**: a checkpoint with no computation or score of its own.
9. **Chop Lab aliasing is simulated in the browser**: when anti-aliasing is off it adds a synthetic tone rather than genuinely folding frequencies (the server's `decimate` does real aliasing).
10. **Hardcoded station targets**: the Caramelize and Chop pages score the player against fixed targets (180 Hz / 0.55 and factor 3) instead of the recipe's own (*Grand Feast* uses 240 Hz / 0.4 and factor 2), so on the Feast the on-screen accuracy disagrees with the server.
11. **Two different ingredient models**: the server synthesises ingredients as instrument-like audio (`backend/app/dsp/instruments.py`) at 22 050 Hz, while the browser uses the mathematical models above. The server score is therefore computed on a different (but parameter-equivalent) dish than the one drawn on screen; only the Filtering Lab shows the server's own signals.
12. **Chop and Caramelize outputs are not drawn into later stations**: their signals are sent to the server and scored, but the Cooking Lab still draws the marinated signal as its input.

8 Glossary

Game hertz	Cycles per normalised one-second window of 401 samples.
Reference dish	The pipeline output for clean ingredients with the recipe's exact parameters.
Stage accuracy	A 0–100 value each station records on the run session.
Prep score	The server's cleaning score for a filtered ingredient.
Session	The server's record of one run (seed, parameters, status).
Attempt	A served dish on the server; the leaderboard's source.